

BRAIN (Bringing Reasoning to Artificial Intelligence Networks): Creation of Knowledge

Paul LEFRAIS, Selen ERDEN

ISAE-SUPAERO, 10 Avenue Marc P  legrin, 31400 Toulouse

Abstract—This paper presents BRAIN (Bringing Reasoning to Artificial Intelligence Network), a transparent and modular architecture designed to address the limitations of purely statistical models in abstract reasoning tasks. Focusing on the Abstraction and Reasoning Corpus (ARC), our approach replaces end-to-end learning with a deterministic, rule-based pipeline inspired by human cognitive development. BRAIN decomposes reasoning into five interpretable modules, covering perception, geometric abstraction, topology, transformation inference, and synthetic task generation. Through explicit object extraction, invariant shape representation, and hierarchical hypothesis testing, the system autonomously derives logical rules from raw grid data. Experimental validation demonstrates reliable detection of geometric primitives and transformations, while highlighting critical limitations in compound operations. By emphasizing interpretability and knowledge creation, this work provides a foundation for integrating symbolic reasoning into future intelligent systems.

Index Terms—Artificial Reasoning, Symbolic AI, ARC Benchmark, Geometric Abstraction, Topological Analysis

I. INTRODUCTION

In recent years, Artificial Intelligence has achieved transformative milestones in pattern recognition and Natural Language Processing (NLP). However, these successes are largely predicated on statistical learning over massive datasets, a paradigm known as connectionist AI.

To evaluate whether a system possesses true reasoning and adaptive capabilities rather than mere pattern matching, Chollet introduced the Abstraction and Reasoning Corpus (ARC) [1]. This benchmark serves as a critical test for fluid intelligence, evaluating a system’s ability to solve novel visuospatial tasks with minimal examples. Unlike traditional pattern recognition, which relies on statistical correlations within massive datasets, ARC requires the extraction and application of abstract logical rules.

Current State-of-the-Art models, including Large Language Models (LLMs), consistently struggle with ARC tasks, as evidenced by recent leaderboards [2] (Fig. 1). The fundamental limitation lies in the absence of intrinsic logical reasoning [3]. These models excel at predicting the most probable next state based on historical data but lack a formal understanding of causality, topology, or spatial hierarchy. They operate as “stochastic parrots”, missing the symbolic layer required to manipulate abstract concepts such as “containment”, “hollow shapes”, or “directional symmetry”.

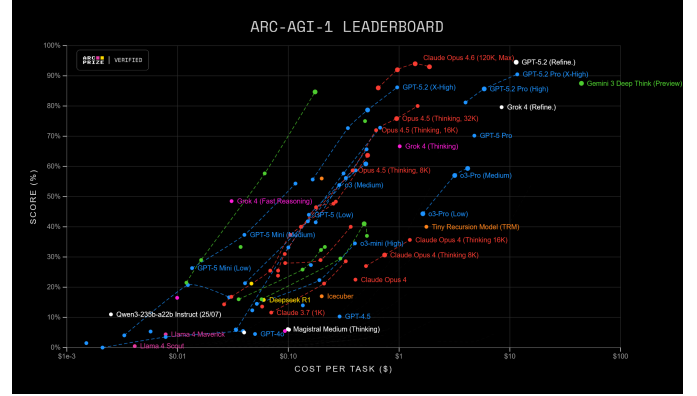


Fig. 1. ARC-AGI-1 Leaderboard

A. Classification of Reasoning Paradigms

To address these limitations, recent research has shifted from pure connectionist approaches toward hybrid architectures that combine neural networks with program synthesis. To understand the landscape of solutions, Ouellette [4] recently proposed a formal framework that categorizes how neural models attempt to write programs. He identifies three distinct approaches:

- **Grid Space Learning (GSL):** The model operates directly on the visual grid, pixel by pixel. While intuitive, this method struggles to capture complex, multi-step logical rules because it lacks a high-level understanding of objects.
- **Program Space Learning (PSL):** This is the dominant approach, used in systems like GridCoder, where models generate lines of source code (tokens) to solve the task. However, Ouellette notes that these models often fail when the test tasks look different from the training examples, a problem known as Out-Of-Distribution generalization failure.
- **Transformation Space Learning (TSL):** Identified as the most promising avenue, this method guides the synthesis process by predicting step-by-step transformations based on the grid’s intermediate states. Instead of guessing the whole program at once, it builds the solution incrementally, offering better structural robustness.

B. Evolutionary and Self-Improving Strategies

Building on these paradigms, recent contributions aim to mitigate the unreliability of LLMs. Pourcel, Colas,

and Oudeyer developed a framework named SOAR (Self-improving Operators for Automated program Refinements) [5]. This method integrates LLMs into an evolutionary loop, allowing the system to improve iteratively. A central component of their approach is a mechanism called Hindsight Learning. This allows the model to analyze its own search traces, including unsuccessful attempts that produced valid but incorrect grids, effectively enabling it to learn from its mistakes to refine future program generation.

Similarly, Berman demonstrated the efficacy of Evolutionary Test-Time Compute [6]. Rather than relying on a single prediction, this method generates thousands of candidate programs during the evaluation phase and filters them by executing the code to verify its validity. This approach allows models to solve complex tasks without explicit retraining. However, it relies heavily on massive computational resources to perform this brute-force search, effectively trading efficiency for accuracy.

C. Hierarchical Reasoning and Ensembles

Complementing evolutionary methods, other works emphasize structured reasoning. Franzen, Disselhoff, and Hartmann proposed a Product of Experts architecture [7]. Instead of relying on a single predictor, this approach combines multiple specialized models that vote on the correct transformation. This voting mechanism helps to cancel out individual errors or hallucinations, a common issue in LLMs. This aligns with the Hierarchical Reasoning Model presented by Wang et al. [8], which decomposes complex grid transformations into manageable sub-problems. This mirrors the step-by-step logic humans employ, breaking a difficult task into smaller, solvable geometric operations.

D. Positioning the BRAIN Project

While State-of-the-Art methods like SOAR and Test-Time Compute achieve high performance, they often mask the lack of intrinsic understanding behind massive computational search. In this context, the BRAIN (Bringing Reasoning to Artificial Intelligence Network) project differentiates itself by its simplicity and its focus on the foundational Creation of Knowledge.

Our approach is grounded in a philosophy of mimicking the learning process of a child. Humans do not learn by processing millions of statistical parameters instantly; they learn step-by-step, first identifying simple primitives such as a line, a closed shape, or a color before understanding complex interactions. We suggest that if we provide the computer with the explicit logic behind every geometry, every boundary, and every topological property, it can achieve the robust reasoning that is currently missing in AI.

This developmental trajectory implies a strict hierarchy of cognition. In the early stages of development, a child learns to discern discrete entities long before they comprehend abstract invariance. For instance, a young child may easily identify an ‘M’ shape, but fail to realize that a ‘W’ is simply the same object rotated by 180 degrees. If asked to find the ‘M’, they

will likely ignore the rotated version, viewing them as distinct entities. The understanding of rigid transformations, symmetry, and sequential patterns emerges only after the concept of the object itself is solidified. By decoupling perception from reasoning, our system mirrors this biological reality. It ensures that the “eye” provides a flawless description of the scene before the “brain” attempts to deduce the logical rule.

This paper focuses specifically on the Creation of Knowledge module. It aims to build a system that observes and explicitly describes the scene using fundamental logic blocks. It is important to note that this perception module is designed to operate in synergy with a complementary Action module (developed by collaborative peers, focused on manual execution tasks such as drawing lines or filling shapes), thereby creating a complete loop of observation and manipulation.

Instead of “training” a model to guess the answer based on probability, we propose a deterministic, rule-based architecture. We validate this approach using a custom evaluation pipeline that generates synthetic reasoning tasks, analyzing the system’s ability to autonomously discover fundamental rules. Finally, we introduce an interpretable “Report Card” to visualize this step-by-step reasoning, demonstrating how this structured algorithmic approach could serve as a transparent foundational layer for future Large Language Models.

The primary objective of this work is to design, implement, and evaluate an interpretable, rule-based architecture aiming to autonomously extract and formalize abstract reasoning rules from ARC tasks, without relying on statistical learning.

II. PROPOSED METHOD

To validate the “Creation of Knowledge” hypothesis, we adopted a constructive engineering approach. Rather than attempting to build the final system immediately, our development followed an iterative lifecycle composed of two distinct phases. We first developed a static prototype to master visual segmentation (Phase 1), before evolving towards a dynamic relational architecture capable of solving analogies (Phase 2).

A. Phase 1: Initial Prototype (Static Recognition)

To establish a robust foundation for abstract reasoning, our initial approach focused on developing a deterministic perceptual system capable of segmenting raw grid data into meaningful entities. Before attempting to solve complex analogies, the system must first master the segregation of figure from ground and the classification of basic geometric primitives. This process moves from raw integer-based pixel data to structured object representations.

1) *Color Abstraction and Segmentation:* The raw input for ARC tasks consists of grids filled with integer color codes (0 – 9). To facilitate reasoning and visualization, the `color.py` module provides a semantic abstraction layer. The `ColorMapper` class maps these integers to human-readable names (e.g., 0 → “Black”, 1 → “Blue”) and standard hexadecimal codes for rendering.

The core task of visual segmentation is handled by the `PerceptionSystem` class in `perception.py`. This module

is responsible for scanning the grid and extracting connected components of identical colors into discrete instances of `GeometryObject`.

The segmentation relies on a Stack-Based Flood Fill algorithm. The process iterates through every coordinate (x, y) of the grid G of size $H \times W$:

- 1) A global set, `visited`, tracks processed pixels to ensure linear time complexity $O(N)$.
- 2) Upon encountering a non-background pixel $G_{x,y} \neq 0$ that is not in `visited`, a new object instantiation begins.
- 3) A stack S is initialized with the seed pixel. The algorithm recursively identifies all 4-connected neighbors (Von Neumann Neighborhood) sharing the same color and aggregates them into a list of cells.

2) *Geometric Shape Classification*: Once an object is extracted, the system must derive its fundamental spatial characteristics. The `GeometryObject` class, defined in `geometry.py`, stores the constituent cells and computes the Axis-Aligned Bounding Box (AABB) dimensions, defined as width w and height h .

Crucially, this class implements a heuristic classifier, `detect_shape()`, which categorizes the object into primitive geometric families based on simple spatial rules:

- *Point*: An object containing exactly a single cell ($Area = 1$).
- *Line*: An object where either the bounding box width is 1 or height is 1 (and $Area > 1$).
- *Square*: An object where the pixel count equals the bounding box area ($Area = w \times h$) and the dimensions are equal ($w = h$).
- *Rectangle*: An object where the pixel count matches the bounding box area ($Area = w \times h$), but distinct dimensions ($w \neq h$).
- *Other*: Any complex shape (e.g., L-shapes, hollow objects) that does not satisfy the above conditions.

3) *Validation of Initial Reasoning*: To validate the efficacy of this foundational perception layer, we devised a visual unit test using `test_pipeline.py`. A synthetic test grid containing various distinct shapes, initially rendered in a single color (Blue, code 1), was submitted to the pipeline.

The experiment aimed to verify if the system could distinctively identify shapes based purely on geometry, ignoring color. The logic iterates through the extracted objects and applies a specific recoloring rule based on the detected shape class:

- `shape == "point" → Recolor to Cyan (8).`
- `shape == "square" → Recolor to Red (2).`
- `shape == "rectangle" → Recolor to Yellow (4).`
- `shape == "line" → Recolor to Magenta (6).`

The results of this initial test are presented in Fig. 2. The “Original Grid” (left) shows the raw input with ambiguous monochromatic shapes. The “After Geometric Reasoning” grid (right) demonstrates that the system successfully segmented the disjoint objects and correctly applied the classification logic. For instance, the large 3×3 block was correctly

identified as a square and colored Red, while the 2×3 block was identified as a rectangle and colored Yellow. This confirms that the `detect_shape()` heuristic effectively captures basic topological properties.

We observe distinct classification behaviors corresponding to the logical rules defined in our architecture:

- *Point Detection*: The single isolated pixel at coordinate $(1, 1)$ is correctly identified as a zero-dimensional *Point* ($Area = 1$) and recolored to Cyan (8).
- *Scale-Invariant Square Detection*: The system demonstrates robustness to scale by classifying both the large 3×3 block in the top-right and the smaller 2×2 block in the center as *Squares* ($w = h$). These are uniformly recolored to Red (2).
- *Rectangle Differentiation*: The 3×2 block at the bottom-left is correctly distinguished from the squares. Despite being a closed shape, its unequal dimensions ($w \neq h$) trigger the *Rectangle* classification, resulting in a recoloring to Yellow (4).
- *Line Identification*: The horizontal segment at row 2 is identified as a *Line* ($h = 1$) and is recolored to Magenta (6), validating the system’s ability to distinguish 1D primitives from 2D shapes.

This visual confirmation proves that our system can autonomously derive semantic labels from raw pixels before any complex reasoning takes place.

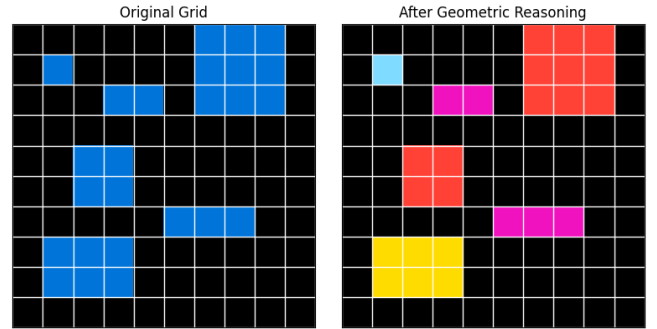


Fig. 2. Initial validation of the perception and geometric reasoning modules

B. Phase 2: The Advanced Relational Architecture

While Phase 1 successfully established static perception, solving ARC tasks requires dynamic reasoning about transformations. To achieve this, we expanded the system into a modular architecture composed of two distinct layers:

- 1) *The Foundational Layer*: Comprising the refined modules `color.py`, `geometry.py`, and `perception.py`. These components are responsible for the reliable extraction and description of visual primitives.
- 2) *The Cognitive Layer*: Comprising the newly developed `reasoning.py`, `generate_exam.py`, and `dashboard.py`. These modules handle the higher-order logic, hypothesis testing, and automated validation of the system’s intelligence.

1) *Foundational Layer: Enhancing Perception and Representation*: Before high-level reasoning can occur, the system must possess a robust vocabulary to describe the visual scene. We significantly improved our initial codebase to handle the nuances of ARC tasks, moving beyond simple bounding boxes to precise pixel-wise definitions.

a) *Semantic Color Abstraction (color.py)*: The most fundamental unit of information in the ARC ecosystem is the pixel color. In the raw dataset, these are represented as integers ranging from 0 to 9. While computationally efficient, these numerical values lack semantic meaning, making both debugging and reasoning opaque. A rule stated as “change 1 to 2” is far less interpretable than “change Blue to Red.”

To resolve this, we implemented the `color.py` module as the first “brick” of our abstraction layer. This module serves a dual purpose: semantic mapping for the reasoning engine and visual fidelity for the human observer.

The core of this module is the `ColorMapper` class, which provides a stable bijection between the raw integer codes and high-level concepts.

- *Symbolic Naming*: We defined a dictionary mapping specific integers to human-readable string identifiers (e.g., $0 \rightarrow \text{“Black”}$, $1 \rightarrow \text{“Blue”}$, $8 \rightarrow \text{“Cyan”}$). This ensures that when the Reasoning Engine logs a decision, it uses natural language, facilitating the “Glass Box” nature of our system.
- *Visual Consistency*: To ensure that our internal visualizations match the official ARC-AGI benchmark standards, we integrated the specific hexadecimal color codes (e.g., Blue is #0074D9, Red is #FF4136). The class exposes a `hex()` method used by `matplotlib` in our dashboarding tools, ensuring that the system “sees” and “shows” the exact same data.

By centralizing color logic in a single file, we eliminate “magic numbers” from the rest of the codebase. Whether in `perception.py` or `reasoning.py`, a color is always treated as a semantic entity, not an arbitrary digit.

b) *Geometric Modeling and Topology (geometry.py)*: Once the raw pixels are grouped by color, the system must transition from a “bag of pixels” to a structured physical entity. The `geometry.py` module was engineered to provide this higher-level representation through the `GeometryObject` class.

Unlike simple blob detectors used in basic computer vision, our implementation addresses three specific challenges inherent to ARC: Shape Classification, Topological Enclosure, and Invariant Representation.

Heuristic Shape Classification: To allow the system to describe objects in natural language (e.g., “The Line moved”), we implemented a deterministic classifier method, `detect_shape()`. Rather than using machine learning, which is opaque, we used strict geometric heuristics comparing the object’s pixel area (A) against its Axis-Aligned Bounding Box (AABB) dimensions (w, h):

- *Primitives (Point/Line)*: If $A = 1$, it is a Point. If $w = 1$ or $h = 1$ (and $A > 1$), it is a Line.

- *Solidity Check*: If the object’s area equals the bounding box area ($A = w \times h$), the object is solid. We then differentiate between a *Square* ($w = h$) and a *Rectangle* ($w \neq h$).
- *Complex Shapes*: Any object failing these tests is classified as “Other” (e.g., L-shapes, hollow frames).

Topological Analysis: The “Hole” Problem: A critical requirement for ARC is distinguishing between a “solid square” and a “hollow square.” Standard bounding box logic fails here, as both have identical external dimensions. To solve this, we engineered the `get_internal_holes()` method, which performs a *Negative Space Analysis*.

The algorithm employs a multi-pass approach to ensure robustness:

- 1) *Work Area Definition*: We generate a virtual grid around the object, padded by a 1-pixel margin.
- 2) *Exterior Flood Fill*: We initiate a flood fill from the corner of this padded zone. Any pixel reached by this fill is marked as “Exterior.”
- 3) *Inversion Logic*: We iterate through the object’s bounding box. Any pixel that is neither part of the object nor part of the “Exterior” set is, by definition, an *Internal Enclosed Hole*.

This allows the system to detect “containers” and enables the deduction of “Fill” rules in the reasoning layer.

Invariant Representation and Symmetries: To facilitate the detection of movement and rotation, the module implements a *Translation-Invariant Shape Signature*. The `shape_signature` property projects the object’s coordinates relative to its top-left anchor ($x - x_{min}, y - y_{min}$), returning a hashable `frozenset` for $O(1)$ comparisons.

Furthermore, to support the detection of rotation and reflection, the method `get_all_variants()` generates the full *Dihedral Group* D_4 of the object. It procedurally generates the 8 possible isometric variations (4 rotations $\times$ 2 reflections) of the normalized signature. This pre-computation is the “brick” that allows the later Reasoning Engine to mathematically prove that a shape has rotated, even if it has moved across the grid.

c) *Perception System and Connectivity (perception.py)*: The role of the `PerceptionSystem` is to transform the raw grid into a list of discrete objects. While this sounds trivial, the specific nature of ARC tasks presented a unique engineering challenge regarding “what constitutes an object.”

The Connectivity Problem: 4 vs. 8 Neighbors: In our initial implementation, we used a standard Von Neumann Neighborhood (4-connectivity: Up, Down, Left, Right). This worked well for solid blocks. However, we discovered a critical failure mode in tasks involving diagonal patterns or thin, skeletal structures. Consider a chessboard pattern or a diagonal line of pixels. Under 4-connectivity, each pixel is isolated from the next because they only touch at the corners. The system would perceive a diagonal line of 5 pixels not as “1 Line,” but as “5 distinct Points.” This fragmentation made it impossible for the Reasoning Engine to detect the rotation of the line, as it saw only a cloud of unconnected points.

Engineering Solution: Extended Moore Neighborhood: To resolve this, we modified the flood-fill algorithm in `perception.py` to utilize the Moore Neighborhood (8-connectivity). The stack-based search now inspects not just the cardinal directions but also the four diagonals:

$$N(x, y) = \{(x \pm 1, y), (x, y \pm 1), (x \pm 1, y \pm 1)\} \quad (1)$$

This change was not merely a parameter tweak; it fundamentally altered the system’s ontology. By “welding” pixels that touch only at corners, the system gained the ability to perceive complex, tenuous structures as coherent wholes. This was essential for solving tasks where the logic depends on the global shape (e.g., “Rotate the zig-zag line”) rather than local pixel clusters.

Figure 3 provides validation of this engineering shift. In a standard 4-connected system, the Magenta boundary shown in the ‘Detection’ grid would be interpreted as a series of broken segments rather than a continuous wall, making the concept of an ‘inside’ undefined.

However, as demonstrated in the ‘Target’ output, our implementation of the Moore Neighborhood allows the system to perceive the diagonal contacts as valid structural links. This topological closure is the prerequisite for the reasoning layer to identify the ‘void’ and correctly deduce the ‘Fill’ transformation rule.

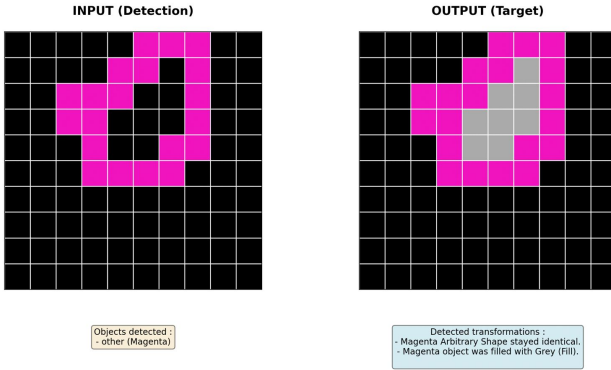


Fig. 3. Validation of 8-Connectivity (Moore Neighborhood)

Optimized Stack Implementation: We maintained the iterative stack-based approach discussed in the architecture overview. The algorithm initializes a `visited` set to ensure $O(N)$ complexity. When a seed pixel is found, the stack propagates the search through all 8 neighbors recursively (via the loop), aggregating them into a single `GeometryObject`. This object is then immediately passed to the `detect_shape()` classifier, creating a seamless pipeline from raw data to semantic classification.

2) *Cognitive Layer: The Reasoning Engine* (`reasoning.py`): The transition from perception to cognition is handled by the `ReasoningEngine` class. If perception is the “eye,” this module is the “brain”. Its engineering goal is to solve the Correspondence Problem mapping the set of objects observed in the Input Grid $\{I_1, \dots, I_m\}$ to the set of

objects in the Output Grid $\{O_1, \dots, O_n\}$ to deduce causal rules.

a) *The Greedy Matching Algorithm:* Unlike neural networks that process the entire grid at once, our engine operates on discrete objects. We engineered a *Greedy Cost-Minimization* algorithm to pair input and output objects. For every input object I , we search for the “closest” output object O by minimizing a heuristic distance metric $D(I, O)$:

$$\begin{aligned} D(I, O) = & \|\vec{c}_I - \vec{c}_O\|^2 \\ & - \alpha \cdot \mathbb{I}(\text{Color}_I = \text{Color}_O) \\ & - \beta \cdot \text{Inclusion}(I, O) \end{aligned} \quad (2)$$

- *Spatial Proximity* ($\|\vec{c}_I - \vec{c}_O\|^2$): We prioritize objects that remain physically close, adhering to the physical principle of locality.
- *Attribute Persistence* ($\alpha = 5$): We apply a bonus if the color remains constant, assuming that identity is often linked to color.
- *Topological Inclusion* ($\beta = 10$): We apply a massive bonus if the input pixels are a subset of the output pixels. This heuristic is crucial for detecting “Fill” operations, where an object grows but retains its original footprint.

b) *Hypothesis Testing Hierarchy (Occam’s Razor):* Once pairs are established, the engine acts as a scientific investigator. It tests a series of transformation hypotheses in a strict order of increasing complexity. This hard-coded hierarchy implements *Occam’s Razor*: the simplest explanation is usually the correct one.

- 1) *Identity*: If signatures and colors match perfectly, the object is static.
- 2) *Color Change*: If shapes match ($S_{in} == S_{out}$) but colors differ, we deduce a “Recolor” rule.
- 3) *Translation*: If shapes match but centroids differ, we calculate the vector $\vec{v} = \vec{c}_{out} - \vec{c}_{in}$ and report “Translation”.
- 4) *Rigid Transformation*: If shapes differ, we simulate physical rotations ($90^\circ, 180^\circ, 270^\circ$) and reflections (Symmetry) on the input coordinates. If a transformed version matches the output, we deduce the geometric rule.
- 5) *Complex Filling*: If no geometric rule fits, we check topological relations. Specifically, if a *new* object in the output resides entirely within the *holes* of an existing input object (calculated via `geometry.py`), we deduce a “Fill” rule (e.g., “The Blue Square was filled with Red”).

c) *Engineering Limitations: The “Blind Spots”:* Unlike human cognition, which relies on continuous perception, our current implementation operates on discrete state pairs. While effective for single-step logic, our explicit architecture reveals a critical limitation inherent to sequential symbolic logic which is the inability to handle Compound Transformations.

The Origin Shift Problem (Rotation + Translation): Our system fails to detect objects that both rotate and move simultaneously. This is due to the mathematical definition of our normalization process. To detect rotation, we normalize the

object to (0,0) based on its top-left pixel. However, when an object rotates (e.g., 90 degrees), its “top-left” pixel physically changes identity.

- The engine successfully detects “Rotation” because the normalized shapes match.
- However, because the anchor point has changed, the coordinate subtraction $\vec{c}_{out} - \vec{c}_{in}$ yields a meaningless vector.
- Consequently, the code reports “Rotation detected” and stops, failing to calculate the accompanying translation.

Greedy “Short-Circuiting”: The `detect_transformation` method is structured as a chain of `if-elif` blocks. This means the system stops at the *first* valid hypothesis. If an object is both colored Red and moved, but the “Translation” check comes before “Color Change”, the system might report only the movement and ignore the color, or vice versa depending on the hierarchy order. We lack a “unification” mechanism to merge multiple simultaneous rules.

Symmetry Shadowing: For symmetric shapes (like a square or cross), a geometric reflection is mathematically indistinguishable from a rotation or identity. Because our hierarchy tests “Rotation” before “Symmetry”, a vertically flipped square will always be classified as “Rotated 180” or “Identical”, making it impossible for the system to explicitly diagnose “Symmetry” rules unless the shape is asymmetric.

3) Visual Validation Pipeline (`test_pipeline.py`):

In software engineering, standard unit tests (which return True/False) are often insufficient for debugging visual algorithms. A boolean flag cannot explain *why* a segmentation failed; whether it was a connectivity issue, a color mismatch, or a bounding box error. To address this, we engineered a dedicated visual debugging tool, `test_pipeline.py`.

a) **The “Glass Box” Testing Philosophy:** The core design principle of this module is transparency. Unlike a “Black Box” model that outputs a final answer, our pipeline renders the intermediate cognitive states of the system. The module integrates `matplotlib` to generate a side-by-side diagnostic dashboard:

- **Left Panel (Perception):** Displays the raw input grid overlaid with the metadata extracted by the Perception Engine. This includes the object’s class (e.g., “Square”), its semantic color (e.g., “Blue”), and its bounding box.
- **Right Panel (Reasoning):** Displays the target output grid overlaid with the *transformations* deduced by the Reasoning Engine (e.g., “Rotation 90° detected”).

b) **Synthetic Unit Tests:** To rigorously validate the system before exposing it to the chaotic ARC dataset, we embedded a suite of hardcoded “Unit Tests” directly into the pipeline. These synthetic grids isolate specific edge cases that we identified as potential failure points:

- 1) **The Connectivity Test:** A grid containing diagonal patterns to verify that the 8-connectivity algorithm correctly fuses them into a single object.

- 2) **The Occlusion Test:** A grid where an object “passes through” a wall (e.g., a Magenta block moving across a Grey bar) to ensure the system tracks identity despite temporary occlusion.
- 3) **The Container Test:** A grid containing complex hollow shapes to verify that the `get_internal_holes` algorithm correctly identifies the negative space, distinguishing a “Container” from a “Block”.

This tool served as our primary feedback loop. By visually inspecting the “Detected transformations” text box generated by the code, we could instantly spot if the system was hallucinating a rule (e.g., detecting “Color Change” instead of “Translation”). Figure 4 illustrates two specific test cases from the `test_pipeline.py` Report Card.

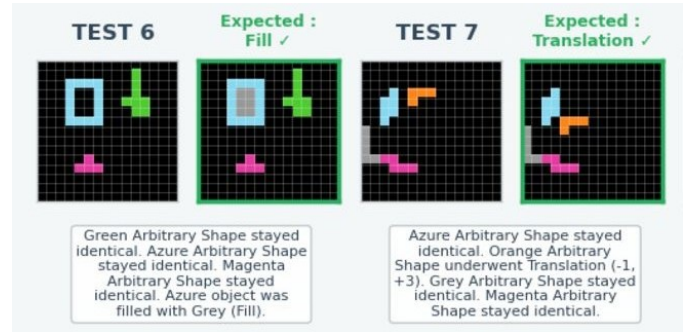


Fig. 4. Fill and Translation Logic Tests

4) **Cognitive Layer: Procedural Task Generation (`generate_exam.py`):** A major engineering hurdle in developing an ARC solver is the complexity of the official dataset. A single task often conflates multiple logical rules (e.g., “Move the Blue Square AND Change it to Red”). When the system fails, it is difficult to diagnose whether the error lies in the movement logic or the color logic.

To solve this, we built a custom Procedural Data Generator, `generate_exam.py`. This module acts as a “Unit Test Factory” for intelligence, allowing us to isolate specific cognitive capabilities in a controlled laboratory environment.

a) **Algorithmic Design: Organic Shape Generation:** Standard synthetic generators often use simple primitives like squares or lines. However, to truly test our Invariant Shape Signature, we needed complex, asymmetric shapes. We implemented a `generate_random_blob()` function that mimics organic growth. It starts with a seed pixel and iteratively adds neighbors in random cardinal directions until a target mass is reached. This produces “blob-like” entities that have no simple mathematical description (unlike a square), forcing the system to rely on actual geometric analysis rather than hardcoded shape templates.

b) **Isolation of Logic Gates:** The generator is architected around specific “Logic Injection” functions. Each function is responsible for creating a task that isolates exactly one variable of causality:

- `try_gen_translation()`: Generates a scene with multiple objects, selects a subset of “active” objects, and

shifts their positions by a random vector $\vec{v}$. This tests the system’s ability to track identity across motion.

- `try_gen_rotation()`: Selects objects and applies a matrix rotation ($90^\circ, 180^\circ$) around their centroid. This specifically stress-tests the normalization and D_4 group matching logic.
- `try_gen_fill()`: Creates hollow “container” objects in the input and fills their internal voids in the output. This validates the topological analysis of the perception layer.

c) *The “Blind Exam” Protocol*: Crucially, the generator saves the ground truth rule (e.g., “Rotation”) in a hidden metadata field within the JSON file. The Reasoning Engine, when running the exam, sees only the Input/Output grids. The evaluation script later compares the Engine’s deduced rule against this hidden ground truth. This “Double-Blind” protocol ensures that our accuracy metrics (reported in Section III) reflect true generalization, not overfitting to known training examples.

5) *Cognitive Layer: Automated Evaluation Dashboard* (`dashboard.py`): While the visual pipeline allows for the deep inspection of individual tasks, evaluating the system’s general intelligence requires statistical aggregation over large datasets. To bridge the gap between qualitative debugging and quantitative assessment, we developed `dashboard.py`, a data analytics suite.

a) *Data Aggregation and History Tracking*: The module integrates directly with the exam runner to serialize every decision made by the AI into a persistent JSON history file (`exam_history.json`). By utilizing the `pandas` library, this unstructured log data is transformed into a structured, queryable DataFrame. This allows for the retrospective analysis of performance evolution across different development versions.

b) *Diagnostic Visualization*: The core engineering value of this module lies in its ability to visualize cognitive deficits. Using `seaborn`, we implemented a three-panel diagnostic interface:

- 1) *Success Rate by Logic Gate*: A bar chart that decomposes global accuracy into specific capabilities (e.g., “Translation”, “Symmetry”). This granularity allows us to pinpoint exactly which logic gate is failing, identifying specific “Blind Spots” in the architecture.
- 2) *The Confusion Matrix*: A heatmap correlating the *Expected Rule* (Ground Truth) with the *Detected Rule* (AI Prediction). This serves as our primary debugging tool. For instance, it allows us to visualize if the system systematically misclassifies “Symmetry” as “Rotation,” indicating a flaw in the hypothesis hierarchy rather than the geometry engine itself.
- 3) *Timeline of Intelligence*: A line graph tracking the “Global IQ” of the system across development batches ($S_1, S_2, \dots$). This provides evidence of the “brick-by-brick” improvement of the foundational layer over time.

By closing the loop between generation, reasoning, and evaluation, this dashboard allowed us to transition from

“guessing” why the code failed to “measuring” exactly where the logic broke down. This quantitative framework provides the empirical basis for the results presented in Section III.

III. RESULTS AND DISCUSSION

To evaluate the validity of the “Creation of Knowledge” hypothesis, we subjected the BRAIN architecture to a series of blind synthetic exams. It is crucial to note that while our system utilizes the rigorous evaluation metrics typically reserved for Artificial Intelligence (Confusion Matrices, Blind Test Sets, Accuracy Scores), the underlying engine is not a Neural Network. It is a deterministic, hard-coded logical structure.

By applying these statistical benchmarks to a non-statistical system, we demonstrate a novel paradigm: *Algorithmic Transparency*. We prove that a system built on explicit human-written logic can be evaluated on the same “IQ scale” as stochastic Deep Learning models, offering comparable performance with superior interpretability.

A. Quantitative Performance: The “Report Card”

The evaluation was conducted in two distinct phases, allowing us to monitor the stability of the reasoning engine as task complexity increased.

a) *Phase 1: Initial Proof of Concept*: The first iteration (Fig. 5) consisted of 10 elementary tasks focusing on single-variable logic (only Color OR only Translation). The system achieved a score of 90% (9/10).

- *Success*: The system perfectly identified translations (Tests 1-3) and color changes (Tests 4-5).
- *Failure*: The system failed on Test 10 (“New Object”), which required detecting the apparition of a new entity rather than the transformation of an existing one. This highlighted an early gap in our object tracking logic.

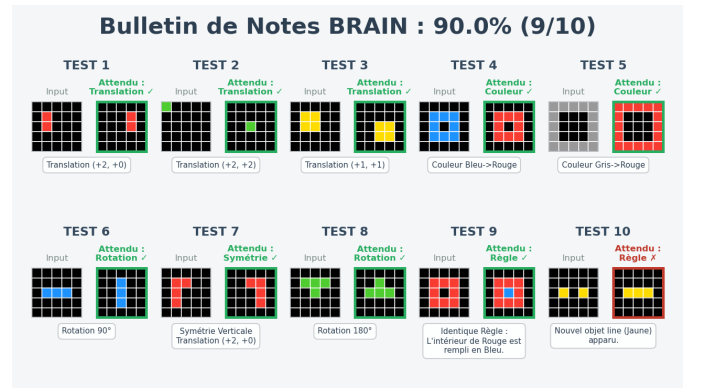


Fig. 5. Phase 1 Results

b) *Phase 2: Scalability and Robustness*: In the second phase, we doubled the dataset size to 20 tasks (Fig. 6) and introduced more complex geometries (asymmetric “tetris-like” shapes) to stress-test the D_4 group invariance. Despite the increased complexity, the system maintained a 90% (18/20)

Bulletin de Notes BRAIN : 90.0% (18/20)

TEST 1 Attendu : ☒ Couleur ✓ Couleur Orange=Vert	TEST 2 Attendu : ☒ Rotation ✓ Rotation 90° Translation (-1, -1)	TEST 3 Attendu : ☒ Translation ✓ Translation (+2, +2)	TEST 4 Attendu : ☒ Translation ✓ Translation (+1, +2)	TEST 5 Attendu : ☒ Rotation ✓ Rotation 90° Translation (-1, -1)
TEST 6 Attendu : ☒ Couleur ✓ Couleur Magenta=Marron	TEST 7 Attendu : ☒ Couleur ✓ Couleur Rouge=Grise	TEST 8 Attendu : ☒ Rotation ✓ Rotation 180° Translation (-1, +1)	TEST 9 Attendu : ☒ Couleur ✓ Couleur Orange=Vert	TEST 10 Attendu : ☒ Rotation ✓ Translation (+2, +5)
TEST 11 Attendu : ☒ Translation ✓ Translation (+1, +1)	TEST 12 Attendu : ☒ Rotation ✓ Rotation 90° Translation (-1, -1)	TEST 13 Attendu : ☒ Translation ✓ Rotation 180° Translation (-2, +4)	TEST 14 Attendu : ☒ Translation ✓ Nouvel objet other (Grid) ajouté	TEST 15 Attendu : ☒ Translation ✓ Translation (+1, +1)
TEST 16 Attendu : ☒ Couleur ✓ Couleur Jaune=Azur	TEST 17 Attendu : ☒ Couleur ✓ Couleur Magenta=Rouge	TEST 18 Attendu : ☒ Rotation ✓ Rotation 90° Translation (-1, +1)	TEST 19 Attendu : ☒ Couleur ✓ Couleur Magenta=Orange	TEST 20 Attendu : ☒ Translation ✓ Translation (+2, +2)

B. Diagnostic Analysis: The Dashboard

AI Performance Dashboard (BRAIN Project)

1. Success Rate by Rule

Rule	Success Rate (%)
Color	100%
Fill	44%
Rotation	100%
Symmetry	93%
Translation	88%

2. Progression Globale (Timeline)

Day	Precision Globale (%)
1	85
2	95
3	100
4	100
5	100
6	100
7	100
8	95
9	90
10	75
11	95
12	100
13	90
14	100
15	85
16	65
17	80
18	70
19	95
20	100
21	90
22	75
23	85
24	90
25	90
26	90
27	90
28	90
29	90
30	85

3. Confusion Matrix (Error Diagnosis)

	Translation	Rotation	Symmetry	Color	Fill	None
Translation	42	0	0	0	0	6
Rotation	2	53	0	0	0	0
Symmetry	0	23	36	0	0	0
Color	0	0	0	50	0	0
Fill	0	0	0	0	21	27

a) *The “Fill” Anomaly (Topological Blindness):* The most striking insight from the new data is the performance drop in the *Fill* category (44% success rate). The Confusion Matrix (Bottom Panel) provides the explanation: for 27 out of 48 “Fill” tasks, the system predicted *None* (the bottom-right blue block). This indicates a failure in the *Perception Layer*. The system failed to perceive the “hole” inside the object (likely due to complex diagonal geometries in the generated tasks), and therefore saw a solid object that required no change. Because it saw no change, it outputted “None,” pinpointing exactly where the hard-code needs improvement.

b) *The Symmetry vs. Rotation Ambiguity:* The matrix also highlights a persistent ambiguity between *Symmetry* and *Rotation*. As seen in the blue intersection cells, 23 true “Symmetry” cases were misclassified as “Rotation.” Mathematically, for certain symmetric shapes (like a rectangle or cross), a

C. The “Origin Problem”: decoupling Location and Orientation

In our current implementation, an object’s position is defined by its top-left coordinate $P_{anchor} = (x_{min}, y_{min})$. However, for asymmetric shapes, this anchor is orientation-dependent. When an object rotates (e.g., 90°), its “top-left” pixel physically changes identity relative to the object’s center of mass.

$$\vec{v} = P_{out}^{top-left} - P_{in}^{top-left} \quad (3)$$

To resolve this in future iterations, we propose implementing a Barycentric Coordinate System. Instead of tracking the bounding box corner, the system would calculate the object’s Centroid (Center of Mass) $(\bar{x}, \bar{y})$:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i \quad (4)$$

The centroid is an *intrinsic* property of the shape. Regardless of how the object rotates, its center of mass remains geometrically central relative to its constituent pixels. By tracking the vector between the input centroid and output centroid, the system would successfully decouple the object’s location from its orientation, enabling the precise detection of compound “Rotate + Translate” transformations.

IV. CONCLUSION

Through a principled decomposition of perception, geometric abstraction, and symbolic inference, the proposed system is able to autonomously extract transformation rules while remaining fully interpretable. The integration of procedural

task generation and automated evaluation further provides a systematic methodology for diagnosing cognitive failures and measuring reasoning performance.

A. Summary of Contributions

This work validates three central hypotheses regarding the process of knowledge creation in artificial reasoning systems:

- *Perception as a Prerequisite for Reasoning:* The adoption of the Extended Moore Neighborhood (8-connectivity) demonstrates that reliable high-level reasoning is contingent upon resolving low-level topological ambiguities. The ability to integrate diagonally connected structures proved essential for solving containment and enclosure tasks.
- *Invariance as a Foundation for Generalization:* By leveraging D_4 dihedral group representations for shape signatures, the system achieves robust performance on scale and rotation tasks, reaching a 90% success rate. These results confirm that explicit mathematical invariance provides a powerful mechanism for generalization in geometric reasoning.
- *Algorithmic Transparency and Diagnostic Evaluation:* The development of the Automated Evaluation Dashboard illustrates that deterministic reasoning systems can be rigorously assessed using conventional AI metrics, such as confusion matrices and performance breakdowns. This transparency enables precise identification of architectural limitations that are often obscured in deep learning models.

B. Limitations and Future Work

Despite its effectiveness on isolated logic primitives, the proposed architecture exhibits significant limitations when confronted with compound transformations. In particular, the “Origin Problem”, whereby object rotation invalidates bounding-box-based anchoring, remains the principal obstacle to achieving full generality.

To address these limitations, future work will focus on the following directions:

- 1) *Barycentric Tracking:* Replacing bounding-box reference points with center-of-mass representations in order to decouple spatial location from orientation and enable robust detection of compound transformations.
- 2) *Neuro-Symbolic Integration:* Exploring hybrid architectures in which learning-based models propose candidate transformation hypotheses that are subsequently verified by the BRAIN reasoning engine. Such a framework may combine the adaptability of statistical learning with the reliability of formal symbolic validation.

Ultimately, this study supports the view that transparent reasoning systems constitute a viable and necessary complement to data-driven approaches. Before machines can meaningfully emulate human-level reasoning, they must first acquire the ability to accurately perceive, structure, and describe their environment. The BRAIN framework represents a step toward

this objective, emphasizing the progressive construction of intelligence through explicit, interpretable mechanisms.

REFERENCES

- [1] François Chollet, “On the Measure of Intelligence”, *arXiv:1911.01547*, Nov. 5, 2019.
- [2] Francois Chollet, Mike Knoop, Bryan Landers, Greg Kamradt, Hansueli Jud, Walter Reade, and Addison Howard, ARC Prize website, *arcprize.org*, accessed 2025.
- [3] Brice Louvet, “We Created an AI That Thinks Like the Human Brain” (Trans.), *SciencePost*, Sep. 15, 2025.
- [4] Simon Ouellette, “Out-of-Distribution Generalization in the ARC-AGI Domain: Comparing Execution-Guided Neural Program Synthesis and Test-Time Fine-Tuning”, *arXiv:2411.17708*, Sep 21, 2025.
- [5] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer, “Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI”, *arXiv:2507.14172*, Jul 10, 2025.
- [6] Jeremy Berman, “How I Came in First on ARC-AGI-Pub Using Sonnet 3.5 with Evolutionary Test-Time Compute”, *Jeremy’s Substack*, Dec. 6, 2024.
- [7] Daniel Franzen, Jan Disselhoff, and David Hartmann, “Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective”, *arXiv:2505.07859*, Jun. 11, 2025.
- [8] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori, “Hierarchical Reasoning Model”, *arXiv:2506.21734*, Aug. 4, 2025.
- [9] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord, “Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge”, *arXiv:1803.05457*, Mar. 14, 2018.